

Lab Assignment 12 (B+ Tree Indexing & Operations)

Department: CSE

Subject: DIS Lab

Code: 22CPP217

Objective

To implement and analyze **B+ Tree indexing mechanisms**, understand node structure, insertion, deletion, and search operations, and experimentally observe how B+ Trees improve database query performance.

Pre-Lab Setup

Run this block to initialize the environment.

-- Reset Environment

```
DROP TABLE IF EXISTS Employees;
```

-- Create Table

```
CREATE TABLE Employees (  
    Emp_ID INT PRIMARY KEY,  
    Emp_Name VARCHAR(50),  
    Salary INT,  
    Department VARCHAR(50)  
);
```

-- Insert Sample Data

```
INSERT INTO Employees VALUES  
(101, 'Aman', 50000, 'IT'),  
(102, 'Riya', 60000, 'HR'),  
(103, 'Kabir', 55000, 'Finance'),  
(104, 'Neha', 70000, 'IT'),  
(105, 'Arjun', 65000, 'HR'),  
(106, 'Simran', 52000, 'Finance'),  
(107, 'Rahul', 72000, 'IT');
```

Q1: Understanding B+ Tree Index Creation

The administrator wants to improve search performance.

- Create an index on the Salary column using B+ Tree indexing.

```
CREATE INDEX idx_salary ON Employees(Salary);
```

- Verify index creation using:

```
SHOW INDEX FROM Employees;
```

- Execute a SELECT query using Salary condition:

```
SELECT * FROM Employees WHERE Salary = 60000;
```

- Observe query execution plan:

```
EXPLAIN SELECT * FROM Employees WHERE Salary = 60000;
```

Q2: B+ Tree Search Operation

- Execute range queries on indexed column:

```
SELECT * FROM Employees WHERE Salary BETWEEN 50000 AND 65000;
```

- Drop the index and run **EXPLAIN** to see the switch from "Index Scan" to "Full Table Scan."

```
DROP INDEX idx_salary ON Employees;
```

```
EXPLAIN SELECT * FROM Employees WHERE Salary BETWEEN 50000 AND 65000;
```

- Recreate the index again.

```
CREATE INDEX idx_salary ON Employees(Salary);
```

Q3: B+ Tree Insertion Mechanics

- Insert new records into Employees table:

```
INSERT INTO Employees VALUES (108, 'Priya', 58000, 'IT');
```

```
INSERT INTO Employees VALUES (109, 'Karan', 75000, 'Finance');
```

- Run SELECT queries again and observe:

```
SELECT * FROM Employees ORDER BY Salary;
```

Q4: B+ Tree Deletion Mechanics

- Delete a record:

```
DELETE FROM Employees WHERE Salary = 52000;
```

- Run query again:

```
SELECT * FROM Employees ORDER BY Salary;
```

Q5: Index on Multiple Columns

- Create a composite index:

```
CREATE INDEX idx_dept_salary ON Employees (Department, Salary);
```

- Execute query:

```
SELECT * FROM Employees  
  
WHERE Department = 'IT' AND Salary > 60000;
```

- Observe performance improvement using EXPLAIN.
-

SQL Syntax Hints

```
<> SQL  
  
-- Create Index  
CREATE INDEX index_name ON table_name(column_name);  
  
-- Drop Index  
DROP INDEX index_name ON table_name;  
  
-- View Index  
SHOW INDEX FROM table_name;  
  
-- Query Execution Plan  
EXPLAIN SELECT * FROM table_name WHERE condition;  
  
-- Insert Data  
INSERT INTO table_name VALUES (...);  
  
-- Delete Data  
DELETE FROM table_name WHERE condition;
```

